c4_support.py — technical

`src/moe_pruning/data/c4_support.py · 169 lines · moe_pruning.data.c4_support`

Purpose

Support-domain filter for C4 documents, de-duplication, counters, and the evaluation-file loader.

Formulas

```
accepted = (n_distinct_phrases >= C4_MIN_KEYWORD_MATCHES) or (url_matched and n_distinct_phrases >= 1)
score = n_distinct_phrases + 0.25 * min(total_occurrences - n_distinct_phrases, 8) + (1.0 if url_matched else 0)
```

Step by step

Classes and functions in source order. Signatures, line ranges, raised exceptions and settings are extracted from the code; numbered steps describe what the code does.

`_phrase_pattern`

```
def _phrase_pattern(phrase: str) -> re.Pattern
```

lines 30-32

1. Split the phrase into words, `re.escape` each, join with `\s+` (any whitespace between words).
2. Wrap in `(?<![A-Za-z0-9])` ... `(?![A-Za-z0-9])` so matches cannot start or end inside a longer word; compile case-insensitive.

`class FilterDecision`

lines 36-42 · dataclass

Result of filtering one document: accepted?, reason, score, matched keywords.

Field	Type	Default
`accepted`	bool	
`reason`	str	
`score`	float	0.0
`matched_keywords`	list[str]	field(default_factory=list)
`keyword_occurrences`	int	0
`url_keywords`	list[str]	field(default_factory=list)

`class SupportDocumentFilter`

lines 45-96

Decides whether a C4 document looks like customer-support content.

`SupportDocumentFilter.evaluate`

```
def evaluate(self, text: str, url: str) -> FilterDecision
```

lines 68-96

Check length, domain, text phrases and URL keywords of one document.

1. Reject `too_short` / `too_long` by character count.
2. If allowed domains are configured, reject when the URL host is neither equal to nor a subdomain of one.
3. Count, per configured phrase, the number of regex matches; collect distinct matched phrases and total occurrences.
4. Collect URL keywords whose pattern matches the URL string.
5. Decide acceptance and score (formulas above). Rejection reason: `url_match_only`, `no_keyword_match` or `insufficient_keyword_matches`.

`normalized_text_hash`

```
def normalized_text_hash(text: str) -> str
```

lines 99-101

Hash of lower-cased, whitespace-normalised text (for de-duplication).

`class FilterStatistics`

lines 105-119 · dataclass

Scanned / accepted / rejected / duplicate counts and rejection reasons.

Field	Type	Default
`scanned`	int	0
`accepted`	int	0
`rejected`	int	0
`duplicates`	int	0
`rejection_reasons`	Counter	field(default_factory=Counter)

`FilterStatistics.to_dict`

```
def to_dict(self) -> dict
```

lines 112-119

Convert to a plain dictionary for JSON reports.

`class Deduplicator`

lines 122-134

Remembers seen texts and URLs.

`Deduplicator.is_duplicate`

```
def is_duplicate(self, text: str, url: str) -> bool
```

lines 127-134

True if the text or URL was seen before (and remember it otherwise).

1. Normalise text (lower-case, collapse whitespace), SHA-1 it.
2. Duplicate if the hash or the (non-empty) URL was seen; otherwise remember both and return False.

`class EvalDocument`

lines 141-145 · dataclass

One evaluation document (id, text, url, timestamp).

Field	Type	Default
`example_id`	int	
`text`	str	
`url`	str	
`timestamp`	str	

`load_eval_documents`

```
def load_eval_documents(path: Path, max_samples: int) -> list[EvalDocument]
```

lines 148-169 · raises FileNotFoundError, ValueError

Read the evaluation JSONL; text is required.

1. Missing file -> `FileNotFoundError` naming the build script.
2. Each non-blank line must parse and contain a non-empty string `text`; `url`/`timestamp` default to ''.
3. Assign sequential `example_id`s; stop at `max_samples`; raise if empty.

Errors raised

Function	Exception	Message (f-string fields in braces)
`load_eval_documents`	FileNotFoundError	Evaluation dataset {path} not found. Run `uv run python scripts/build_c4_support_eval.py`.
`load_eval_documents`	ValueError	No evaluation documents in {path}
`load_eval_documents`	ValueError	{path}:{line_number}: record has no 'text'
`load_eval_documents`	ValueError	{path}:{line_number}: invalid JSON ({exc.msg})